

INSPER · AULA 05 · 27/08/2026 · UNIDADE 3: MACHINE LEARNING APLICADO À SEGURANÇA

Como máquinas aprendem a detectar intrusões?

Aprendizado supervisionado, features de tráfego de rede e Random Forest, do sinal heurístico ao CICIDS2017

DISCIPLINA

Cibersegurança Aplicada com IA

FORMATO

Aula expositiva · Roteiro 2 em duplas

```
aluno@insper:~$ python meu_dataset.py
--dupla 07
111.482 fluxos · 58% BENIGN · 79
features
    rf.fit(Xtr, ytr).score(Xte, yte)
0.989 # será?
```

Roteiro de *hoje*

- | | | |
|----|------------------------------------|---|
| 01 | Aprender com supervisão | Rótulos, classificação × regressão, de log bruto a matriz de features |
| 02 | Treino, teste e vazamento | Por que separar, e os três vazamentos que produzem modelos ótimos e inúteis |
| 03 | Datasets e fluxos de rede | CICIDS2017, a 5-tupla e o que um fluxo conta sobre o atacante |
| 04 | Dados sujos + Random Forest | Defeitos do dataset, armadilhas plantadas e 200 árvores votando |
| 05 | Roteiro 2 | Duplas · cada dupla, um dataset · itens a) a l) |

CONTEXTO

De onde viemos, para onde *vamos*

DE ONDE VIEMOS

- ◆ **Roteiro 1 (Aula 2):** sinais de log extraídos à mão, com regras
- ◆ **Aula 4:** como um IDS enxerga tráfego
- ◆ **Hoje:** os sinais viram features de um classificador, primeiro em autenticação, depois em tráfego de rede real

AO FINAL DESTA AULA VOCÊ DEVE SABER

- ◆ Explicar o fluxo dados → features → treino/teste → modelo → avaliação
- ◆ Reconhecer vazamento de dados e como o **Pipeline** evita
- ◆ Criar features de tráfego a partir de intuição de rede e limpar um dataset real
- ◆ Ler uma matriz de confusão multiclasse e explicar cada erro em termos de rede

APRENDER COM SUPERVISÃO

O problema do Roteiro 1, *revisitado*

```
# Roteiro 1, regra escrita por humano
alerta = (falhas ≥ 10) & \
        (usuarios_inexistentes ≥ 1)

# e se o atacante for lento?
# e se o usuário esqueceu a senha?
```

- ◆ Cada limiar (10? 1?) foi escolhido por alguém, **uma vez**
- ◆ Combinar 3 sinais é fácil; combinar 8 é **impossível de manter**
- ◆ Toda exceção vira mais uma cláusula
- ◆ A regra **não melhora sozinha** com novos dados

APRENDIZADO SUPERVISIONADO INVERTE O PROBLEMA

Em vez de escrever a regra, você dá **exemplos rotulados** (este IP era ataque, aquele não) e um algoritmo encontra a fronteira que melhor separa os dois grupos, usando todos os sinais ao mesmo tempo.

APRENDER COM SUPERVISÃO

Classificação e *regressão*

HOJE E AULAS 8, 9, 10

Supervisionado

Cada exemplo tem um rótulo (y). O modelo aprende $f(X) \approx y$.

Precisa de dados rotulados, **caros em segurança**.

A DISTINÇÃO QUE IMPORTA

Classificação vs. regressão

Classificação: y é categoria: benigno / brute_force; BENIGN / DDoS / PortScan...

Regressão: y é número: bytes esperados, tempo até falha.

Detecção de intrusão é **quase sempre classificação**.

AULA 7

Não supervisionado

Sem rótulo: encontra estrutura ou anomalias. Isolation Forest, clustering.

Útil quando não sabemos como o ataque se parece.

PERGUNTA AO GRUPO

Detectar fraude em cartão: classificação ou regressão? E prever quantos alertas o SOC terá amanhã?

APRENDER COM SUPERVISÃO

De log bruto a *matriz de features*

```
Aug 25 02:13:44 srv sshd[2201]: Failed password for invalid user admin from 10.42.7.19 port 51522 ssh2
Aug 25 02:13:46 srv sshd[2201]: Failed password for invalid user root from 10.42.7.19 port 51530 ssh2
Aug 25 02:13:47 srv sshd[2203]: Failed password for oracle from 10.42.7.19 port 51544 ssh2
```

▼ agregar por ip_origem, janela de 24 h

IP_ORIGEM	TENTATIVAS_TOTAL	FALHAS	USUARIOS_DISTINTOS	INTERVALO_MEDIO_SEG	FRACAO_NOTURNA	USUARIOS_INEXISTENTES	LABEL
10.42.7.19	187	181	31	1.4	0.91	14	brute_force
10.8.120.3	6	1	1	2804.7	0.27	0	benigno
10.66.2.240	11	9	1	310.2	0.12	1	benigno ← esqueceu a senha

Linhas = exemplos (uma por IP). Colunas = features (X). Última coluna = rótulo (y). ip_origem NÃO é feature: é identificador.

O que faz uma feature ser *boa* em segurança

BOAS FEATURES

Comportamento

- ◆ Codificam **comportamento**, não **identidade** (razão de falhas, não IP)
- ◆ Invariantes ao que o atacante controla facilmente (ele muda o IP; não muda o padrão de tentativas)
- ◆ Calculáveis em produção com a mesma definição usada no treino

ARMADILHAS

O que evitar

- ◆ Identificadores (IP, hostname, timestamp bruto)
- ◆ Features que 'vazam' o rótulo (ex.: 'alerta_gerado' pelo IDS)
- ◆ Escalas muito diferentes sem padronização (intervalo em s vs. fração em $[0,1]$)

E O RÓTULO (LABEL)?

Rótulo é caro

- ◆ Vem de incidente confirmado, honeypot, sandbox ou dataset público
- ◆ Rótulo errado em 5% dos exemplos ensina o modelo a errar 5% das vezes, e o pior: com confiança
- ◆ Daqui a pouco: CICIDS2017, ataques executados de propósito em laboratório, por isso os rótulos

2

Treino, teste e *vazamento*

O modelo não é bom por acertar o que já viu.

TREINO, TESTE E VAZAMENTO

Por que separar *treino e teste*

ORIGEM

Dataset rotulado

3.160 IPs



SPLIT

70 / 30

`train_test_split, stratify=y`



TREINO · 2.212

`fit()`

O modelo só vê este lado



TESTE · 948

`predict()`

→ métricas

```
from sklearn.model_selection import train_test_split
```

```
Xtr, Xte, ytr, yte = train_test_split(  
    X, y, test_size=0.30,  
    stratify=y, random_state=2026)
```

- ◆ **stratify**: mesma proporção de ataque nos dois lados
- ◆ **random_state**: reprodutível, você e eu obtemos o mesmo split
- ◆ O teste é aberto **UMA vez, no fim**

Overfitting: o modelo decora o treino (acurácia 100% no treino, 85% no teste). **Underfitting**: simples demais para capturar o padrão (ruim nos dois).
Diagnóstico completo na Aula 9.

TREINO, TESTE E VAZAMENTO

Vazamento de dados: o erro que produz modelos *ótimos e inúteis*

VAZAMENTO 1

Pré-processar antes do split

O scaler calcula média e desvio com o teste dentro. O modelo 'conhece' estatísticas que não deveria.

Solução: Pipeline.

VAZAMENTO 2

Feature que é o rótulo disfarçado

'alerta_ids', 'bloqueado_pelo_fw', 'ticket_aberto' foram gerados **DEPOIS** de saber que era ataque.

VAZAMENTO 3

Duplicatas entre treino e teste

A mesma linha nos dois lados: o modelo 'acerta' porque já viu. O CICIDS2017 tem **milhares de duplicatas**.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

modelo = Pipeline([('scaler', StandardScaler()), ('clf', LogisticRegression(max_iter=1000))])
modelo.fit(Xtr, ytr)      # o scaler só vê o treino
modelo.predict(Xte)      # aplica no teste as estatísticas do treino
```

TREINO, TESTE E VAZAMENTO

Árvore de decisão: o modelo que *escreve regras*

```
from sklearn.tree import DecisionTreeClassifier, plot_tree

arv = DecisionTreeClassifier(max_depth=3,
                             random_state=2026)

arv.fit(Xtr, ytr)

# árvore aprendida no dataset de aquecimento:
# usuarios_distintos ≤ 3.5 ?
#   ├ sim: usuarios_inexistentes ≤ 1.5 ? benigno : ataque
#   └ não: ataque
```

COMO ELA APRENDE

- ◆ Em cada nó, testa todas as features e limiares
- ◆ Escolhe o corte que mais 'purifica' os grupos (Gini)
- ◆ Repete até a profundidade máxima
- ◆ Resultado: uma regra if/else **legível por humano**

A ÁRVORE NÃO REINVENTOU A REGRA DO R1, ENCONTROU OUTRA

Regra manual (falhas ≥ 10 e usuários inexistentes ≥ 1): acurácia 96%, deixa passar 20% dos ataques lentos. Árvore de profundidade 3: **acurácia 99,8%**, com um primeiro corte que nenhuma dupla escreveu. Discuta no roteiro (item c): qual das duas você colocaria em produção?

TREINO, TESTE E VAZAMENTO

Métricas mínimas para hoje: acurácia e *matriz de confusão*

	PREVISTO BENIGNO	PREVISTO ATAQUE
REAL BENIGNO	VN = 778	FP = 2 alerta falso
REAL ATAQUE	FN = 10 o erro que importa	VP = 158

Regressão logística, conjunto de teste (948 IPs)

98,7%

acurácia (936 / 948)

10

ataques que passaram (FN)

LER A MATRIZ, NÃO SÓ A ACURÁCIA

- ◆ **FN (falso negativo):** ataque classificado como benigno, o pior erro num IDS
- ◆ **FP (falso positivo):** alerta falso, desgasta o analista
- ◆ Recall do ataque = $VP / (VP + FN) = 158/168 = 94\%$
- ◆ Precisão do ataque = $VP / (VP + FP) = 158/160 = 99\%$
- ◆ **Acurácia mente quando a classe é rara:** com 85% de benigno, 'sempre benigno' já tem 85%

Daqui a pouco: 98,9% de acurácia no split aleatório vão esconder um **recall de 0,16 no Bot** quando o teste for o futuro. Curvas PR, PR-AUC e limiar: Aula 9 e Roteiro 3.

3

Datasets e *fluxos de rede*

De onde vêm os rótulos, e o que um fluxo conta sobre o atacante.

DATASETS E FLUXOS DE REDE

CICIDS2017: uma semana de tráfego em *Frederickton*



2,8 M

fluxos rotulados

80

features por fluxo

CICFLOWMETER

15

tipos de ataque

+ BENIGN

~80%

dos fluxos são BENIGN

O QUE USAMOS HOJE: O DATASET REAL, UM RECORTE POR DUPLA

Pool de 182 mil fluxos dos 8 CSVs originais, 79 features do CICFlowMeter, 15 rótulos. **meu_dataset.py** gera para cada dupla ~111 mil fluxos (58% BENIGN) com random_state e classe difícil próprios. **Três armadilhas foram plantadas no pool**, vocês vão caçá-las no item e.

DATASETS E FLUXOS DE REDE

A unidade de análise mudou: agora é o *fluxo*

5-tupla: (10.0.0.5 : 51522) → (192.168.1.10 : 22) TCP ⇒ um fluxo = todos os pacotes dessa conversa

Volume

Total Fwd / Bwd Packets
Total Length of Fwd / Bwd
Flow Bytes/s, Packets/s

Forma dos pacotes

Fwd / Bwd Packet Length Max,
Mean
Min / Max / Mean Packet Length
Header Length

Tempo

Flow Duration
Flow IAT Mean / Std / Max
Fwd / Bwd IAT, ritmo de cada lado

Protocolo

Destination Port
SYN / ACK Flag Count
Fwd PSH Flags
Init_Win_bytes_forward

DATASETS E FLUXOS DE REDE

Como cada ataque deve aparecer nas *features*?

CLASSE	O QUE O ATACANTE FAZ	HIPÓTESE NAS FEATURES	MEDIANA OBSERVADA (REAL)
DDoS (LOIC)	Inunda o servidor com pedidos mínimos	Pacotes de ida minúsculos, resposta grande, porta 80	7 B ida · 1.934 B volta · 2 s · porta 80
PortScan	1 pacote SYN por porta, centenas de portas	1 pacote, duração ~0, milhares de portas distintas	1 pkt · 48 µs · 41.667 pkt/s · 1.020 portas
SSH-Patator	Tenta senhas em série na porta 22	Muitos pacotes de ~80 B, fluxo de vários segundos, porta 22	19 pkts · 80 B · 9,9 s · porta 22
DoS slowloris	Abre conexões e as mantém abertas sem completar	Fluxos longuíssimos, quase sem bytes	3 pkts · 8 B · 100 s · 0,2 pkt/s
Bot (Ares)	Máquina infectada fala com C2 em HTTP	Fluxos pequenos, ritmo periódico, porta 8080	3 pkts · 6 B · 71 ms · porta 8080, parece HTTP normal
BENIGN	Navegação, DNS, SSH interativo, uploads	Tudo entre os extremos	2 pkts · 37 B · 31 ms · 11 mil portas

DATASETS E FLUXOS DE REDE

Engenharia de features: colocar *conhecimento de rede* na tabela

```
b['razao_fwd_bwd'] = b['Total Fwd Packets'] / \
                    (b['Total Backward Packets'] + 1)
b['bytes_por_pacote'] = (fwd_bytes + bwd_bytes) / \
                        (fwd_pkts + bwd_pkts)
b['porta_bem_conhecida'] = (b['Destination Port'] < 1024)
b['syn_sem_ack'] = (SYN == 1) & (ACK == 0)
```

POR QUE NÃO DEIXAR O MODELO DESCOBRIR

- ◆ Modelo mais simples e mais rápido
- ◆ Menos dados necessários
- ◆ Explicação em termos que o analista entende

CADA FEATURE É UMA FRASE DE REDE

- ◆ Ida sem volta → scan ou flood
- ◆ Bytes por pacote → SYN vazio vs. payload
- ◆ SYN sem ACK → handshake que nunca completa
- ◆ Porta < 1024 → serviço padrão

O QUE EVITAR

- ◆ Features que dependem do rótulo
- ◆ Identificadores (IP de origem, timestamp absoluto)
- ◆ Selecionar features olhando o teste (vazamento)

4

Dados sujos e *Random Forest*

O CICIDS2017 tem defeitos; 200 árvores erram menos que uma.

DADOS SUJOS + RANDOM FOREST

Quatro defeitos conhecidos, e três *armadilhas plantadas*

1 • Infinity em Flow Bytes/s e Packets/s

Fluxos de duração 0 (1 pacote) → divisão por zero. ~150 valores no seu recorte; ~4.900 no dataset completo. Tratamento: substituir por NaN e remover, **nunca por zero** (zero significa o oposto).

2 • NaN esparsos

Poucas dezenas no recorte. Se a coluna for essencial, remover a linha; se não, imputar com mediana, **sempre calculada no treino**. E 8 colunas constantes (Bulk) + 1 duplicada (`Fwd Header Length.1`): fora.

3 • Linhas duplicadas

~300 mil no dataset completo. Se ficarem, caem em treino E teste → o modelo 'acerta' porque decorou. `drop_duplicates()` antes do split, mas ele NÃO pega o que difere em 1 µs.

4 • Nomes e rótulos malformados

' Destination Port' com espaço inicial; 'Web Attack \x96 Brute Force' com byte não-ASCII. `columns.str.strip()` e `encoding='latin-1'`.

DADOS SUJOS + RANDOM FOREST

Random Forest: muitas árvores, cada uma vendo *um pedaço*



POR QUE FUNCIONA

- ♦ Uma árvore profunda decora o treino (alta variância)
- ♦ Árvores diferentes erram em pontos diferentes
- ♦ A média de muitos erros independentes é menor que cada erro

O QUE VOCÊ GANHA

- ♦ Robustez sem ajustar hiperparâmetros
- ♦ `feature_importances_`: quanto cada feature reduziu impureza
- ♦ Funciona com features em escalas diferentes (sem scaler) · multiclasse nativo

O QUE VOCÊ PERDE

- ♦ A regra legível da árvore única
- ♦ Tempo: 200 × mais custo de treino e predição
- ♦ Importância ≠ causalidade (features correlacionadas dividem o crédito)

DADOS SUJOS + RANDOM FOREST

O resultado do roteiro: 98,9% de acurácia. *Depois, 95,5%.*

CLASSE	RECALL ALEATÓRIO	RECALL TEMPORAL	N TESTE
Bot	0,964	0,156	585
DoS slowloris	0,994	0,797	988
DoS Hulk	0,991	0,888	1.989
DoS Slowhttpstest	0,995	0,890	955
Web Attack - XSS	0,332	0,321	196
DDoS	0,993	0,988	2.565
PortScan	0,990	0,988	2.437
SSH-Patator	0,998	0,977	606
BENIGN	0,998	0,998	18.749

Random Forest (200 árvores), dupla de referência do gabarito. Split temporal: dentro de cada (dia, rótulo), primeiros 70% por ordem_captura treinam, últimos 30% testam.

0,989

acurácia · aleatório

0,955

acurácia · temporal

O QUE O SPLIT ALEATÓRIO ESCONDEU

- ◆ **Bot: 0,96 → 0,16.** No fim da campanha os fluxos do bot duram 17× mais (canal C2 estabelecido). O treino nunca viu essa fase.
- ◆ DoS lentos caem 10-20 p.p.: a ferramenta muda o ritmo ao longo do ataque.
- ◆ Aleatório mede 'reconhece o que já viu'; temporal mede '**reconhece o que vem depois**'.

DADOS SUJOS + RANDOM FOREST

Este modelo funcionaria na rede do Inspêr *hoje?*

A REDE MUDOU

Tráfego de 2026

- ◆ Majoritariamente TLS/QUIC em 2026, tamanhos e ritmos diferentes
- ◆ Serviços, proporções de portas, MTU, perfis de uso
- ◆ Tráfego benigno real é mais variado que o B-Profile

OS ATACANTES MUDARAM

Ferramentas novas

- ◆ LOIC e Patator têm 'assinatura' de tráfego própria
- ◆ Ferramentas de 2026 randomizam ritmo, tamanho e porta
- ◆ O modelo aprendeu **as ferramentas, não a intenção**

NÃO HÁ RÓTULOS

Sem feedback

- ◆ No Inspêr ninguém diz 'este fluxo é ataque'
- ◆ Sem rótulo, sem retreino, sem medir recall
- ◆ Precisa de honeypot, incidentes confirmados ou feedback do SOC

*Modelo treinado em dataset público é **ponto de partida, não produto**. Aula 20 (drift) e Roteiro 5 tratam de como perceber quando ele parou de funcionar.*

ROTEIRO 2 · DUPLAS · CADA DUPLA, UM DATASET

Roteiro 2 em *uma página*

ITEM	O QUÊ	ENTREGÁVEL NO NOTEBOOK
a	meu_dataset.py --dupla	código, random_state, classe difícil
b	Aquecimento: X/y + regra do R1	matriz da regra; linhas que ela errou
c	Split + árvore d=3	métricas + plot_tree + comparação
d	Regressão logística (Pipeline)	coeficientes; experimento sem scaler
e	Diagnóstico + caçada às armadilhas	3 armadilhas com código, contagem, tratamento
f	Separabilidade por feature única	tabela classe × feature; histograma da classe difícil
g	Features testadas	≥ 4 derivadas, cada uma com teste
h	RF aleatório vs. temporal	recall por classe nos dois splits
i	Importâncias	top-15
j	Classe difícil	feature nova testada, com evidência
k	Parecer contra a produção	5 argumentos com números próprios
l	Split seg-qui → sex	matriz da sexta; classes nunca vistas

REGRAS

- ◆ random_state da dupla em tudo
- ◆ Todo achado com código e número
- ◆ Números conferidos contra o dataset da dupla ($\pm 0,01$)
- ◆ **alertas_ids** no modelo final: -2,0

ENTREGA

- ◆ **R2_<código>.ipynb** executado de ponta a ponta
- ◆ Relatório PDF ≤ 6 páginas com Q1-Q8 e Anexo B
- ◆ Prazo: a combinar em sala

PARA LEVAR

Cinco ideias para *levar*

- ◆ No R1 você escreveu a regra. Hoje o modelo aprende uma, e acha cortes que ninguém escreveu.
- ◆ Features codificam comportamento; identificadores decoram. A unidade de análise (IP, fluxo) é decisão de engenharia.
- ◆ Treino e teste separados, estratificados, com seed. Pré-processamento dentro do Pipeline. O CICIDS2017 pune quem inverte.
- ◆ Datasets de intrusão são gerados em laboratório: rótulo confiável, realismo limitado. E toda coluna que 'prevê tudo' merece a pergunta: de onde veio?
- ◆ 98,9% no split aleatório viraram 95,5% no temporal, e o Bot foi de 0,96 para 0,16. **Avaliação honesta é a do futuro, não a da amostra.** Aulas 8-10 e 20 começam aqui.

Referências: GÉRON (2023) caps. 1-3, 7 • CHIO & FREEMAN (2018) cap. 2 • SHARAFALDIN et al. (2018) • ENGELEN, TROIA & VERBEEK (2021)